

Delta Lake MERGE Implementation

Incremental Data Processing — Assignment Summary

Objective

Perform incremental data processing using Delta Lake. The goal is to load an existing customer dataset, clean it, simulate a batch of new/incoming data, and apply a MERGE operation so that existing customer records are updated and new customer records are inserted — all in a single ACID-safe transaction.

Datasets

File	Description
customer_master.csv	Existing customer database (contains a duplicate row and a few missing values, used for cleaning)
customer_incremental.csv	Today's incoming data: 3 updates to existing customers, 2 brand-new customers

Meaning: customer_id 102, 104, 107 already exist → UPDATE. customer_id 111, 112 are new → INSERT.

Workflow

CSV → Data Cleaning (remove duplicates, handle nulls) → Delta Table → MERGE (Upsert) → Validation → Final Output

Concepts Used

- Delta Lake
- MERGE (Upsert)
- ACID transactions
- Incremental Loading
- Apache Spark (PySpark)

Implementation Notes

The notebook uses PySpark to read and clean the raw CSV files (null handling with fillna, duplicate removal with dropDuplicates). The cleaned data is then written into a genuine Delta Lake table (Parquet data files plus a _delta_log transaction log). The MERGE step uses whenMatchedUpdate / whenNotMatchedInsert semantics to update the 3 existing customers and insert the 2 new customers in one transaction.

Result

- Duplicate row in the master dataset was detected and removed.
- Missing age/city values were filled in during cleaning.
- Existing customer records (102, 104, 107) were updated with the incoming values.
- New customer records (111, 112) were inserted.

- Post-merge validation confirmed the row count is correct and there are zero duplicate customer_id values.
- Delta transaction log shows each write (initial load, then merge) as a separate, auditable version.

See notebooks/delta_scd_assignment.ipynb for the full, executed implementation and screenshots/ for step-by-step execution evidence.